

NURobotics Robotics Software Course

Week 1: Introduction to C++ & Introduction to ROS

Who Am I?

- Casey Goyette
- PlusOne Master's student in Electrical & Computer Engineering w/ Concentration in Hardware & Software for Machine Intelligence
- VEX U software lead
- Co-ops/Internships
 - Electrical Engineering @ ColdSnap
 - Firmware Engineering @ Fleet Robotics
 - Embedded Software @ NK Labs
 - Embedded Software @ Bose



Course Logistics

- Head to #course-roles to assign yourself the robotics software class role and get access to the #robotics-software-course channel
- This course will be held every Thursday from 6:30PM – 8:00PM
 - 10/1, 10/8, 10/15 – Richards 236
 - 10/22 – Dodge 050
 - 10/29 – Richards 253
 - 11/5 – Richards 254
 - 11/12 – Richards 236
 - 11/19 – Shillman 135
 - 12/3 – Richards 254
- Any room/time changes will be communicated in #robotics-software-course
- Course format:
 - Each class will start with a lecture and end with a project (that will likely need to be finished outside of class)
 - Lecture room will be open from 6PM-6:30PM for people to finish projects and ask questions

Intro to C++

C++ Resources

- Compiler Explorer: <https://godbolt.org/>
 - Online C++ code editing and compilation
- CPP Reference: <https://en.cppreference.com/>
 - Online C/C++ reference with explanations and interactive examples for most C/C++ features

Constants

- Declare constant variables with the “const” keyword
- Lets the compiler optimize code better
- Communicates to other developers that a value can't change
- Variables marked with “constexpr” are also constant variables
 - “constexpr” declares variables and functions that can be evaluated at compile time

```
int my_variable = 10;  
const int my_constant = 34;  
  
my_variable += 5; // Works fine  
  
my_constant += 5; // Won't compile
```

Namespaces

- Groups entities together in a namespace scope
- Prevents naming conflicts
- Entities declared outside of a namespace belong to the global scope

```
namespace LibraryA {  
    void printStatus() {  
        // Do something  
    }  
}
```

```
namespace LibraryB {  
    void printStatus() {  
        // Do something  
    }  
}
```

```
LibraryA::printStatus();  
LibraryB::printStatus();
```

Namespaces

- Common namespaces we'll see
 - `std`
 - The C++ standard library namespace
 - `rclcpp`
 - The ROS C++ client library namespace
- Namespaces can be nested
 - `cv::aruco::drawDetectedMarkers(...);`
- Namespaces can be aliased
 - `namespace nru = nurobotics::utils;`
- “using namespace” lets you use names from a specific namespace without writing the namespace name every time
 - This can become problematic if there are conflicting names in imported namespaces

```
using namespace std;  
cout << "Hello World!" << endl; // No std:: needed before cout or endl
```

“auto” Keyword

- Variables in C++ have an associated type that can't change

```
int x = 10;  
x = "Hello World!\n" // Won't compile
```

- “auto” can be used to tell the compiler to deduce the type for us

```
auto x: int = 10; // Type "int" deduced from initialization  
x = "Hello World!\n" // Still won't compile
```

- THIS DOES NOT MAKE C++ DYNAMICALLY TYPED!!!!

Header Files

- C++'s solution to separating code into multiple files while allowing resources from one file to be pulled into another file
- Two types of files in C++:

Header Files:

- *.h or *.hpp
- Declare symbols
 - Classes/functions/global variables

```
Void Func();
```

Implementation Files:

- *.cpp
- Define symbols
 - Function definitions
 - Global variable initializers

```
Void Func() {  
    // implementation  
}
```

Header Files

- Including headers
 - `#include <rclcpp/rclcpp.hpp>`
 - `<>` indicates an external library
 - `#include "MyHeader.hpp"`
 - `"` indicates part of current project
- Statements starting with `"#"` are preprocessor directives
 - A preprocessor is a tool that processes source code before compilation, performing tasks such as text substitution, including files, and conditional compilation
- `#include` inserts the contents of the included file into the current file before compilation
 - This is why we generally include header files rather than implementation files, otherwise we could end up with multiple definitions of symbols

Header Files

- Header guards are necessary to prevent accidentally indirectly including the same header file twice, which would lead to duplicate symbol declarations

```
#ifndef MY_HEADER_HPP
#define MY_HEADER_HPP

// All code here

#endif
```

Containers (C-Style Arrays)

```
int arr[3] = {[0]=5, [1]=6, [2]=7};  
std::cout << arr[1]; // prints "6"
```

- Generally better to use containers provided by standard library rather than raw arrays

Containers (std::array)

- std::array (defined in header <array>)

```
std::array<int, 3> arr = {[0]=5, [1]=6, [2]=7};
```

```
std::cout << arr[1]; // prints "6"
```

```
std::cout << arr.at(1); // prints "6"
```

```
std::cout << arr.at(4); // throws exception
```

```
std::cout << arr.size(); // prints "3"
```

Containers (std::vector)

- std::vector (defined in header <vector>)

```
std::vector<int> vec = {[0]=11, [1]=12, [2]=13};
```

```
std::cout << vec[1]; // prints "12"
```

```
std::cout << vec.at(n: 1); // prints "12"
```

```
std::cout << vec.size(); // prints "3"
```

```
vec.push_back(x: 14); // appends "14"
```

Containers (Iterators)

- Iterators represent a specific position inside specific container
- *.begin() returns the iterator for the first element in a container
- *.end() returns the iterator for one position past the last element

```
vec.erase(position: vec.begin() + 3); // erases third element
```

Range-Based For Loops

- Classic for loop:

```
std::vector<double> v = {[0]=11, [1]=12, [2]=13};

for (int i = 0; i < v.size(); ++i) {
    std::cout << v[i] << std::endl;
}
```

- Range-based for loop:

```
std::vector<double> v = {[0]=11, [1]=12, [2]=13};

for (double elem : v) { // could also use const auto elem
    std::cout << elem << std::endl;
}
```

Strings

- Character variables can hold a single character

```
char var = 'N';
```

- Raw containers aren't convenient for text

```
std::vector<char> v = {[0]='H',[1]='e',[2]='l',[3]='l',[4]='o'};
```

- `std::string` is purpose-built for text

```
std::string s = "Hello World";
```

- Notice that single quotes (") are used for character literals, and double quotes (") are used for string literals

Strings

- `std::string` sometimes behaves like a container

```
std::string s = "Hello";  
s.empty(); // false  
s.size(); // 5  
s.at(n: 0); // 'H'  
s[1]; // 'e'  
s.erase(pos: 3); // erases third character  
s.clear(); // erases all characters
```

Strings

- `std::string` also has many string-specific functions
- Comparison

```
std::string a = "Apple";  
std::string b = "Banana";  
a < b; // true
```

- Converting to/from strings

```
std::to_string(val: 1.23); // "1.23"  
std::stoi(str: "100"); // 100 as int  
std::stod(str: "2.5"); // 2.5 as double
```

- `std::string::npos`
 - Special constant in `std::string` that represents an invalid index
 - Can sometimes mean “the end of the string”

```
s.substr(pos: 5, n: std::string::npos); // substring from the 5th character to the end  
s.find(s: "Hi"); // returns std::string::npos if not found
```

Accessing Object Members

- Dot syntax for plain objects
 - `obj.member()`;
- Arrow syntax for pointers
 - `ptr->member()`;

Intro to ROS

What is ROS?

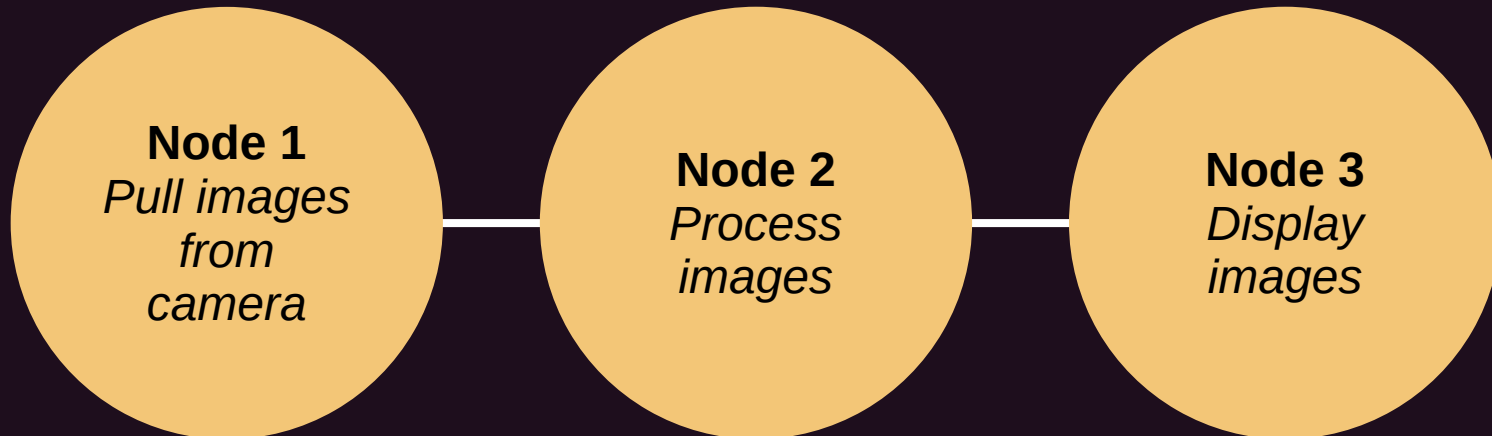
- ROS = Robot Operating System
 - Not actually an operating system
- Collection of tools, libraries and standards for building robot software
- Provides a common way for robot components to communicate
- Allows for the reuse of software instead of building everything from scratch
- Supports hardware drivers, sensors, actuators, visualization, simulation, and algorithms
- Focused on collaborative robotics development
 - Different parts of a robot can be developed as different programs
- Open source and supported by a large robotics community

ROS Resources

- <https://www.ros.org/>
 - Main website with info about ROS
- <https://docs.ros.org/>
 - Documentation for core ROS concepts and tools
- <https://robotics.stackexchange.com/>
 - Q&A site for learning about and working with robotics
- <https://index.ros.org/>
 - Documentation for specific packages with version-specific info

How Does ROS Work? (Nodes)

- Instead of having one big program that handles all behavior for a robot, ROS uses many smaller programs which each handle a piece of the problem
- Each small program is called a node, and an individual robots will contain many nodes
- Each node has a specific purpose
- Different node implementation can be changed or swapped without having to recompile the whole code base or other nodes being affected



Connecting Nodes

- Nodes are able to remain separate and independent by clearly defining the connections supported by each node
- Any two nodes that support the same kind of connections can be hooked together at runtime
- The three types of ROS connections
 - Topics
 - Services
 - Actions

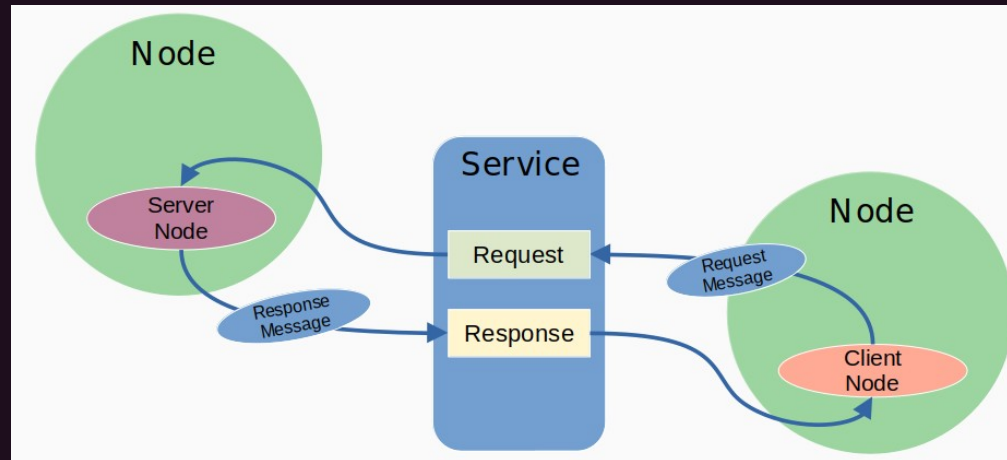
Topics

- Simplest and most common type of connection
- Sends one-way information between nodes
- Nodes publish messages to a named topic
- Other nodes subscribe to that topic
- Supports one-to-one, one-to-many, man-to-many, and many-to-one communication
- Useful for tasks like streaming sensor data or sending motion commands



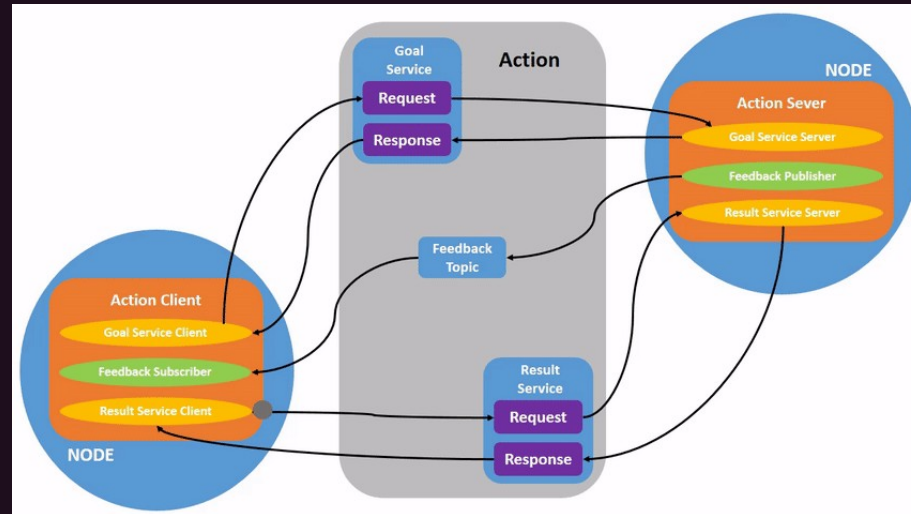
Services

- Request/response communication between nodes
- A client sends a request to a service
- A server processes the request and sends a response
- The client typically waits for the response
- Useful for when data needs to be queried or a node wants to provide an interface for a calculation (path planning, etc.)



Actions

- Request/feedback/result communication
- A client sends a goal to an action server
- Action server provides periodic feedback to the client while working
- Client receives a final result when complete
- Useful for asynchronous, long-running tasks (navigation, high-level behavior, etc.)



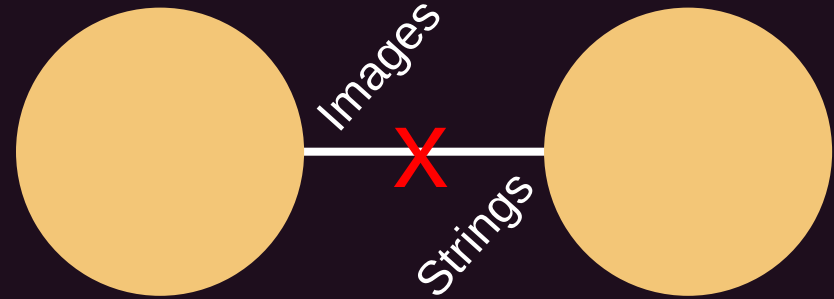
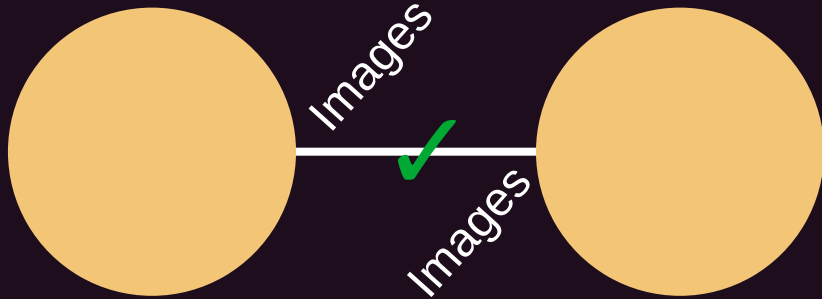
Messages

- Messages are structured data
 - Every message has a type that defines the layout of its data
- Message types are made of fields
 - Fields are like variables and can be expressed as common data types or other message types
 - GPS interface message example:

```
std_msgs/Header header
float64 latitude
float64 longitude
float64 altitude
float64 hdop
float64 utm_easting
float64 utm_northing
string utc
uint8 zone
string letter
```

Connections + Messages

- Each connection has a specific message type associated with it
- There is exactly one type of message used for each part of the communication
 - Topics have one associated message type, and all messages sent across a topic must be of that type
 - Services and Actions have multiple message types (one for requests, one for response, one for feedback, etc.)



ROS Resource Names

- ROS uses a hierarchical system to identify resources like nodes, topics, services, etc.
- Structured somewhat like file paths
 - `/camera1/images`



ROS Resource Names

- No limit to how deep namespaces can be
- Nodes don't have to be at the top (or part of a topic's name at all)
 - Node names identify computation
 - Topic names identify communication channels
- For example, image a project with multiple robots, each with multiple cameras. To get the images from color camera on the front robot named Terry:
 - Topic: `/frank/camera/front/color/images`
 - Node advertising topic: `/frank/camera/front/driver`
- Namespaces can be defined and remapped at runtime, so the complete name of every resource might not be known when writing the code
 - “~” is the node's private namespace and will be replaced with whatever the node's namespace is at runtime
 - `~/exposure`
 - `/terry/camera/front/driver/exposure`
 - `/frank/camera/front/driver/exposure`

Using ros2 & Running Nodes
